

Security Analysis: Owner Slashing in Commit-Reveal Lotteries

Abstract

This report evaluates the slashing mechanics within the Commit-Reveal Lottery smart contract. It provides a concise analysis of scenarios where the owner is penalized (“slashed”) appropriately to protect users, contrasted with edge cases where they are slashed unfairly due to network or EVM constraints.

1 Protocol Mechanics & The Slashing Rule

The contract uses a commit-reveal scheme for randomness. To prevent the owner from manipulating the outcome by refusing to reveal the secret, a slashing mechanism is enforced.

The Rule: If the owner fails to call `revealAndDraw()` within **250 blocks** (≈ 50 minutes) of the `targetBlock`, anyone can call `slashOwner()`. The owner’s collateral is distributed to ticket buyers.

2 Slashing Analysis Matrix

The following table breaks down when the slashing mechanism works exactly as intended, versus when it unfairly penalizes an honest owner.

Scenario	Classification	Analysis & Outcome
Outcome Manipulation	Appropriate	Cause: Owner computes the winner off-chain, realizes they didn't win, and intentionally withholds the reveal. Outcome: Owner is slashed. Participants are protected and refunded.
Protocol Abandonment	Appropriate	Cause: Owner loses private keys, goes offline, or neglects to run the reveal script. Outcome: Failsafe triggers. Owner is slashed, preventing user funds from being locked forever.
Gas Spikes & Stuck Tx's	Tail Risk (Operational)	Cause: A sudden gas spike occurs. If the owner's bot underprices the gas fee, the transaction gets stuck in the mempool. Outcome: If the owner fails to bump the gas within 50 mins, the deadline expires and they are slashed.
Builder Censorship	Theoretical Tail Risk	Cause: Dominant block builders (PBS) blacklist the owner's address (e.g., OFAC). Outcome: While highly unlikely to cause a full 50-minute delay, heavy censorship could dangerously run down the clock.

3 Risk Assessment of Edge Cases

While the aforementioned edge cases represent critical vulnerabilities, it is important to contextualize their likelihood within the current Ethereum ecosystem:

- **Prolonged Congestion:** A continuous, unbroken 50-minute period of extreme network congestion is highly uncommon. However, a bot submitting a transaction with an outdated `maxFeePerGas` is a very common operational hazard.
- **Censorship Impact:** A full 50-minute delay caused purely by builder censorship

is highly improbable. There is a sufficient distribution of non-censoring builders and vanilla validators to ensure transaction inclusion well before the deadline expires.

Verdict: In smart contract auditing, risk is evaluated as *Likelihood* $\times$ *Impact*. Although the strict probability of these events occurring within the exact 50-minute window is low, their impact is catastrophic (complete loss of collateral). Forcing the protocol's security to rely on a tight liveness assumption is a significant architectural anti-pattern.

4 The Core Issue: EVM Limitations

Why not just increase the deadline to 24 hours to prevent unfair slashing?

- **EVM 256-Block Limit:** The EVM opcode BLOCKHASH only stores the hashes of the most recent 256 blocks.
- **The Exploit:** If the window is larger than 256 blocks, a malicious owner could simply wait. `blockhash(targetBlock)` would return 0. The owner could then predictably calculate the winner with zero entropy.

5 Recommendations

To fix the heavy operational risk placed on the owner while maintaining user security, the randomness source must be decoupled from the EVM BLOCKHASH limit.

1. **Use EIP-4399 (PREVRANDAO):** Replace `blockhash` with `block.prevrandao` (the Beacon Chain's RANDAO mix). This removes the 256-block limit, allowing the reveal window to be safely extended to 24+ hours.
2. **Implement Chainlink VRF:** Remove the manual commit-reveal process entirely. Use a decentralized oracle to fulfill randomness requests automatically, eliminating the need for owner collateral and slashing entirely. (as done in our chainlink vrf variant)